

Estrutura de Dados

Kickoff do LogiRota e Revisão Aplicada

[Baixe aqui os exemplos da aula](#)



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

O Problema: Malha de Entregas

O projeto do semestre é o LogiRota, um núcleo de roteirização. Existem pontos de coleta e entrega, ruas que os ligam com distâncias distintas, pedidos que chegam ao longo do dia e a necessidade de decidir por onde passar. Nenhuma lista, pilha ou fila responde sozinha à pergunta central: qual é o melhor caminho.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Revisão: O TAD como Contrato

Um Tipo Abstrato de Dados declara quais valores existem e quais operações são válidas, sem comprometer-se com nenhuma implementação. Enquanto o contrato for respeitado, a representação interna pode ser trocada livremente. Essa liberdade será usada várias vezes no semestre, quando estruturas ingênuas forem substituídas por versões eficientes.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Revisão: Fila e Ordem de Chegada

A fila atende ao princípio do primeiro a entrar, primeiro a sair. É a estrutura correta quando a ordem de processamento deve reproduzir a ordem de chegada, como ocorre com os pedidos do dia. Escolher fila não é preferência estilística, e sim consequência direta da regra do problema.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Revisão: Pilha e Desfazer

A pilha atende ao princípio do último a entrar, primeiro a sair. Desfazer uma operação significa reverter a mais recente, e não a mais antiga, de modo que a ordem de desfazimento é o inverso exato da ordem de execução. Esse comportamento é exatamente o que a pilha oferece.



Prof. Everaldo Graciliano Souza Ribeiro

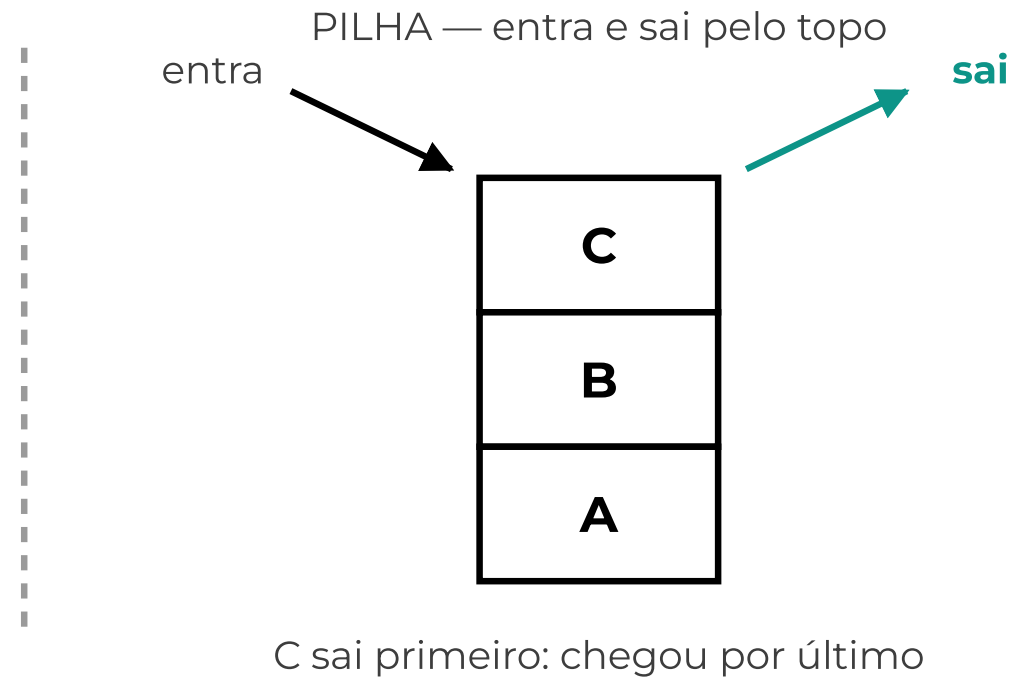
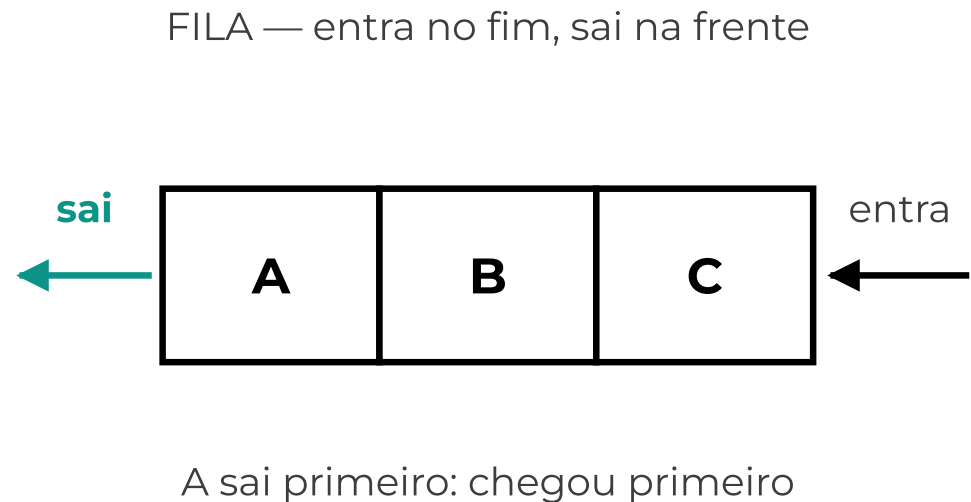
Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Fila e Pilha: Comportamentos Opostos



Mesma entrada A, B, C. **A ordem de saída é oposta** — e é isso que decide qual usar.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Por Que Listas Não Bastam

Para descobrir se existe caminho entre dois pontos seria necessário examinar combinações sucessivas de trechos, e a lista não guarda quem se liga a quem. A informação que falta não é o conjunto de pontos, e sim o conjunto de ligações entre eles. Representar ligações exige outra família de estruturas.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

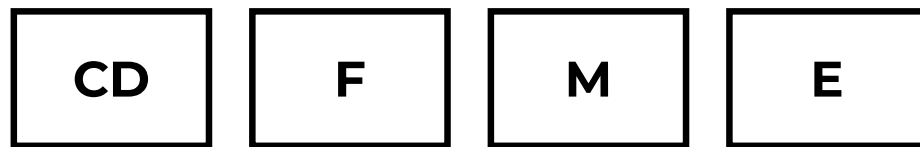
Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

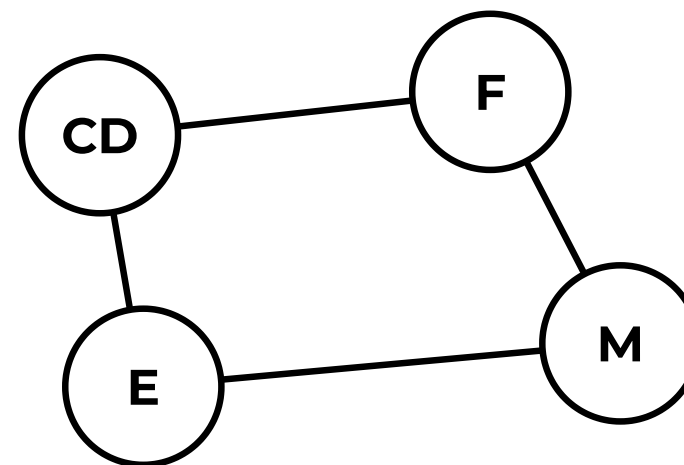
Guardar Pontos ou Guardar Ligações

hoje: lista de pontos



sabe QUAIS pontos existem

a partir da Aula 06: grafo



sabe QUEM se liga a QUEM

A lista responde **quais pontos existem**. Só o grafo responde **por onde dá para ir**.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

everaldoribeiro01@gmail.com

<https://www.linkedin.com/in/everaldoribeiroads>

Panorama: Árvores e Grafos

A árvore organiza elementos em hierarquia, com um único caminho entre a raiz e cada elemento, o que a torna adequada a índices e classificações. O grafo admite qualquer conjunto de ligações, inclusive ciclos, o que o torna adequado a redes e malhas. O semestre percorre primeiro a árvore, depois o grafo.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

Como o Projeto Será Construído

A disciplina reserva um tempo semanal para conceito e três para construção, revisão e medição. Toda estrutura em estudo será implementada manualmente antes de qualquer biblioteca pronta, e cada operação declarará seu custo em comentário. O projeto final não é trabalho separado: é o estado do repositório na última semana.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

TAD Ponto

```
# Módulo 1 aplicado: o TAD declara valores e operacoes; a posicao interna  
# (x, y) e decisao de implementacao e fica oculta atras do prefixo _.
```

```
class Ponto:  
    def __init__(self, nome, bairro, x, y):  
        self._nome = nome  
        self._bairro = bairro  
        self._x = x  
        self._y = y  
  
    def distancia_ate(self, outro):  
        # Na Aula 06 este valor vira o peso da aresta entre dois vertices.  
        dx = self._x - outro._x  
        dy = self._y - outro._y  
        return round((dx * dx + dy * dy) ** 0.5, 2)
```



Fila de Pedidos: FIFO

```
# A ordem de atendimento reproduz a ordem de chegada.
class FilaDePedidos:
    def __init__(self):
        self._itens = []

    def enfileirar(self, pedido):
        self._itens.append(pedido)          # custo constante

    def desenfileirar(self):
        if self.vazia():
            raise IndexError("fila de pedidos vazia")
        # pop(0) desloca todos os elementos restantes: custo proporcional a n.
        return self._itens.pop(0)
```



Pilha de Desfazer: LIFO

```
# Desfazer reverte a operacao mais recente, nunca a mais antiga.
class PilhaDeOperacoes:
    def __init__(self):
        self._itens = []

    def empilhar(self, operacao):
        self._itens.append(operacao)           # custo constante

    def desempilhar(self):
        if self.vazia():
            raise IndexError("nada a desfazer")
        # pop() no fim nao desloca nada: custo constante.
        return self._itens.pop()               # compare com pop(0) da fila
```



As Três Estruturas em Uso

```
fila = FilaDePedidos()
for destino in PONTOS[1:]:
    fila.enqueue(destino)

desfazer = PilhaDeOperacoes()
while not fila.vazia():
    atendido = fila.dequeue()           # sai na ordem de chegada
    desfazer.empilhar(atendido)

while not desfazer.vazia():
    print(desfazer.desempilhar())       # sai na ordem inversa

# A malha ainda e uma lista de pontos: nao ha ligacoes entre eles.
```



Atividades

Exercício 1

Por que a fila de pedidos é fila e a pilha de desfazer é pilha? Troque as duas e diga o que quebra.

Exercício 2

Qual das duas estruturas tem uma operação de custo proporcional ao número de elementos? Qual operação é essa?

Exercício 3

Que informação a lista de pontos não guarda e que impede calcular uma rota?



Desafio extra

Opcional

Quem quiser ir além do combinado nesta semana:

1. Acrescentar ao Ponto um método que informe se dois pontos estão no mesmo bairro.
2. Fazer a fila devolver o tempo médio de espera dos pedidos atendidos.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.



Prof. Everaldo Graciliano Souza Ribeiro

Especialista em Desenvolvimento de Sistemas Para a Internet

Universidade Federal de Viçosa

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>